Nios® V Processor Reference Manual

Updated for Intel® Quartus® Prime Design Suite: **23.1**

Online Version
Send Feedback

683632
2023.05.26

Contents

1. Overview

The Nios® V processor is a soft processor core with the following characteristics:

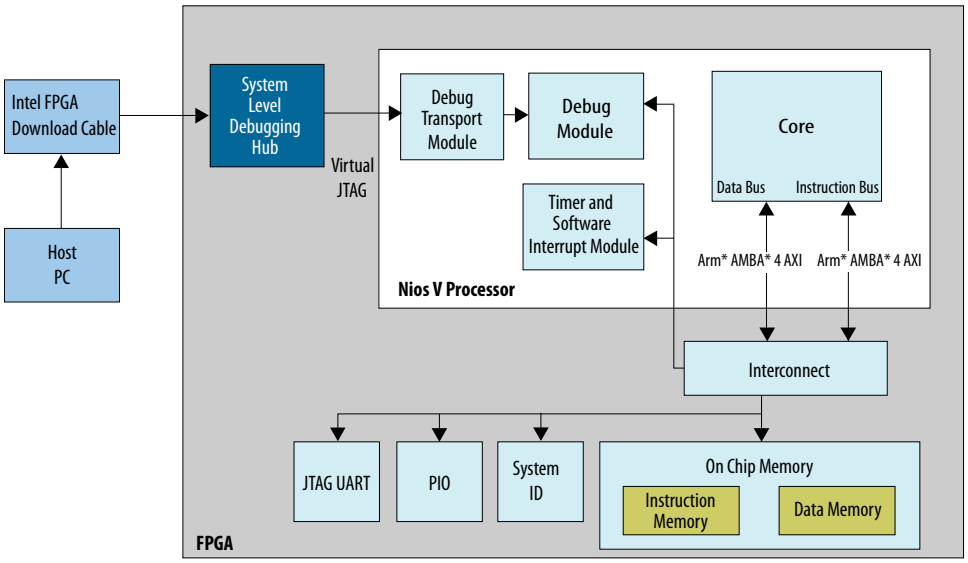
- Designed for Intel® FPGA devices and developed based on RISC-V* specification
- Does not include peripherals or the connection logic to the external devices
- Includes only the circuits required to implement the Nios V processor architecture

The following table lists the available variants:

Table 1. Nios V Processor Variants

Processor Core	Variant
Nios V/m	Microcontroller
Nios V/g	General-purpose processor

Figure 1. Nios V Processor System on Intel FPGA



Related Information

- RISC-V Specification
- Nios V/m Processor on page 5
  More information about the Nios V/m processor (Microcontroller).
- Nios V/g Processor on page 27
  More information about the Nios V/g processor (General-purpose processor).

1.1. Intel Quartus® Prime Software Support

Nios V processor build flow is different for Intel Quartus® Prime Pro Edition software and Intel Quartus Prime Standard Edition software. Refer to *AN 980: Nios V Processor Intel Quartus Prime Software Support* for more information about the differences.

Related Information

AN 980: Nios V Processor Intel Quartus Prime Software Support

2. Nios V/m Processor

The Nios V/m processor is a microcontroller core developed by Intel based on the RISC-V RV32IAZicsr instruction set and supports the functional units described in this document.

Related Information

Overview on page 3

2.1. Processor Performance Benchmarks

Table 2. Nios V/m Processor Performance Benchmarks in Intel FPGA Devices for Intel Quartus Prime Pro Edition Software

FPGA Used	f_{MAX} (MHz)	Logic Size (ALM)	Architecture Performance	
			DMIPS/MHz Ratio	CoreMark/MHz Ratio
Intel Cyclone® 10	252.13	1645	0.567	0.400
Intel Arria® 10	279.43	1662		
Intel Stratix® 10	323.46	1797		
Intel Agilex® 7	397.512	1763		

Table 3. Benchmark Parameters for Intel Quartus Prime Pro Edition Software

Parameter		Settings/Description
Intel Quartus Prime seed		Maximum performance result are based on 10 seed sweep from Intel Quartus Prime Pro Edition software version 23.1.
Device speed grade		Fastest speed grade from each Intel FPGA device family.
Defined peripherals		• Nios V/m processor core (without debug module). • 128 KB on-chip memory for the instruction and data bus. • JTAG UART Intel FPGA IP.
Toolchain	Version	• riscv32-unknown-elf-gcc (GCC) version 12.1.0 • CMake Version: 3.23.2
	Compiler configuration	• Compiler flags: `-O3` • Assembler options: `-Wa -gdwarf2` • Compile options: `-Wall -Wformat-security -march=rv32ia -mabi=ilp32`

Table 4. Nios V/m Processor Performance Benchmarks in Intel FPGA Devices for Intel Quartus Prime Standard Edition Software

FPGA Used	f_{MAX} (MHz)	Logic Size	Architecture Performance	
			DMIPS/MHz Ratio	CoreMark/MHz Ratio
Intel Cyclone IV E	139.63	2958 (LE)	0.572	0.402
Intel Cyclone V	162.1	1302.9 (ALM)		
Intel Arria V	198.65	1308.5 (ALM)		
Intel Arria V GZ	266.52	1263.9 (ALM)		
Intel Stratix V	329.38	1280.3 (ALM)		
Intel Cyclone 10 LP	134.44	2946 (LE)		
Intel Arria 10	325.41	1283.1 (ALM)		
Intel MAX® 10	105.89	2997 (LE)		

Table 5. Benchmark Parameters for Intel Quartus Prime Standard Edition Software

Parameter		Settings/Description
Intel Quartus Prime seed		Maximum performance result are based on 10 seed sweep from Intel Quartus Prime Pro Edition software version 22.3.
Device speed grade		Fastest speed grade from each Intel FPGA device family.
Defined peripherals		• Nios V/m processor core • 4 KB on-chip memory for the instruction bus • 4 KB on-chip memory for data bus • Avalon® Memory-Mapped Pipeline Bridge
Toolchain	Version	• riscv32-unknown-elf-gcc (GCC) version 11.2.0 • CMake Version: 3.23.2
	Compiler configuration	• Compiler flags: `-O3` • Assembler options: `-Wa -gdwarf2` • Compile options: `-Wall -Wformat-security -march=rv32ia -mabi=ilp32`

Intel uses the same Intel Quartus Prime design example for maximum performance benchmark(fMAX) and logic size benchmarks. However, the compiler settings are different for each benchmarks:

- fMAX benchmark: `superior_performance_optimized_placement_effort`
- Logic size benchmark: `area_aggressive`

Note: Results may vary depending on the version of the Intel Quartus Prime software, the version of the Nios V processor, compiler version, target device and the configuration of the processor. Additionally, any changes to the system logic design might change the performance and LE usage. All results are generated from design built with Platform Designer.

2.2. Processor Pipeline

The Nios V/m processor employs a five-stage pipeline.

Table 6. Processor Pipeline Stages

Stage	Denotation	Function
F	Instruction fetch	• PC+4 calculation • Next instruction fetch • Pre-decode for register file read
D	Instruction decode	• Decode the instruction • Register file read data available • Hazard resolution and data forwarding
E	Instruction execute	• ALU operations • Memory address calculation • Branch resolution • CSR read/write
M	Memory	• Memory and multicycle operations • Register file write • Branch redirection
W	Write back	• Facilitates data dependency resolution by providing general-purpose register value.

The Nios V/m processor implements the general-purpose register file using the M20K memory blocks. The processor takes one processing cycle to read from an M20K location. Therefore, the F-stage initiates register file reads so general-purpose register values are available in D-stage.

Writing to the M20K location takes two processing cycles. Therefore, the M-stage initiates writes to a general-purpose register. If there is a dependency to resolve, the M-stage carries forward the value to the W-stage.

The core resolves data dependencies in the D-stage. Operands can move from register file read or E-stage, M-stage, or W-stage.

Reasons for the pipeline stalling:

- Data dependency—if the source operand is not available in D-stage, instruction in D-stage and F-stage stalls until the operand becomes available. The scenario can happen if destination general-purpose register of load or multicycle instruction in E-stage or M-stage is the source for instruction in D-stage.
- Resource stall—if a memory operation or multicycle is pending in M-stage, the instructions in preceding stages stalls until M-stage completes the instruction.

2.3. Processor Architecture

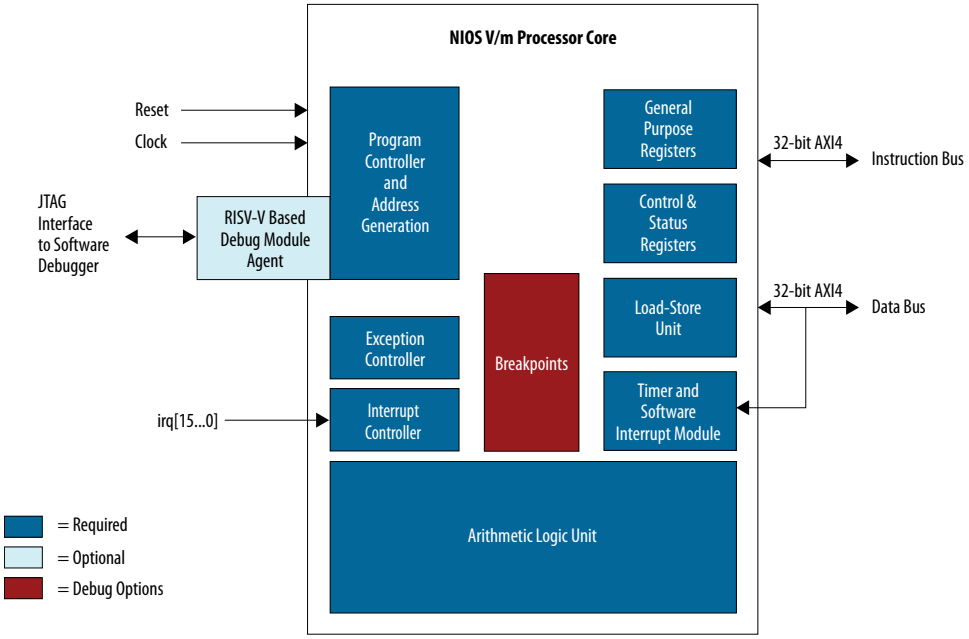
The Nios V/m processor architecture describes an instruction-set architecture (ISA). The ISA in turn necessitates a set of functional units that implement the instructions.

The Nios V/m processor architecture defines the following functional units:

- General-purpose register file
- Arithmetic logic unit (ALU)
- Control and status registers (CSR)
- Exception controller
- Interrupt controller

- Instruction bus
- Data bus
- RISC-V based debug module

Figure 2. Nios V/m Processor Core Block Diagram



2.3.1. General-Purpose Register File

Nios V/m processor implementation supports a flat register file. The register file contains thirty-two 32-bit general-purpose integer registers. Nios V/m processor implements the general-purpose register using M20K memories, which do not support two read ports. Hence, Nios V/m processor duplicates the register files so that two different source registers for an instruction are available in a single cycle. After performing ALU operations, the processor core writes the same result to the destination register in both memories.

2.3.2. Arithmetic Logic Unit

The arithmetic logic unit (ALU) operates on data stored in general-purpose registers. ALU operations take one or two inputs from registers and store the result back into the register.

Table 7. Fundamental Data Operations of the ALU

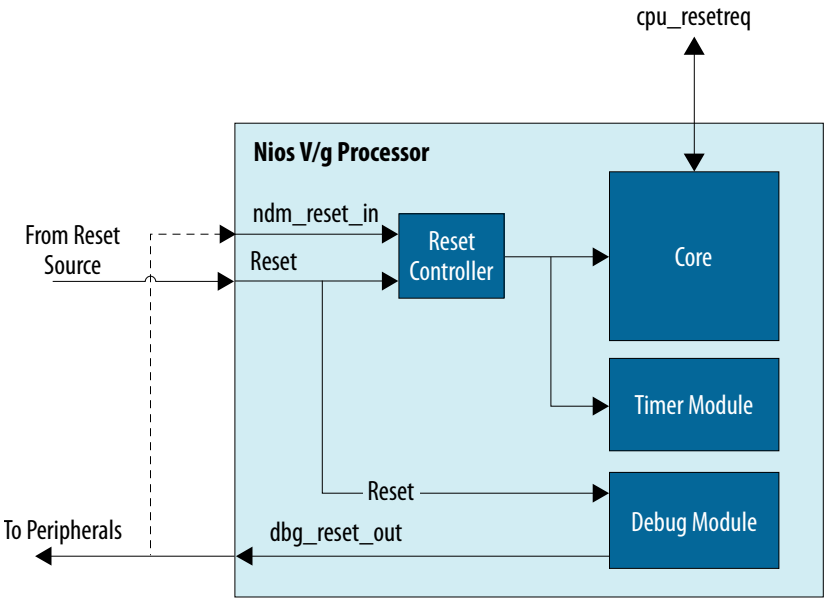
Category	Description
Arithmetic	Addition and subtraction on signed and unsigned operands.
Relational	Equal, not-equal, greater-than-or-equal, and less-than relational operations (==, != >=, <).
Logical	AND, OR, NOR, and XOR logical operations.
Shift	Logical and arithmetic shift operations.

For load and store instructions, the Nios V/g processor uses the ALU to calculate the memory address. For conditional control transfer instructions, Nios V/g processor uses the relational operations in the ALU to determine if the processor takes or leaves the branch..

2.3.3. Reset and Debug Signals

Interface	Type	Description
`reset`	Reset	A global hardware reset input signal that forces the Nios V processor to reset immediately.
`dbg_reset_out`	Reset	An optional reset output signal which appear after you enable both **Enable Debug** and **Enable Reset from Debug Module** parameters. • This reset output signal is triggered by the JTAG debugger or `niosv-download -r` command. • You can connect this reset output signal to the following input signals: — To the `ndm_reset_in` input to reset the core and the timer module. — To the reset input signal of other components as needed.
`ndm_reset_in`	Reset	An optional reset input signal which appear after you enable both **Enable Debug** and **Enable Reset from Debug Module** parameters. • You can use this signal to trigger reset controller to reset the core and the timer module. • The reset controller synchronizes the reset (hard reset) and `ndm_reset_in` signals.
`cpu_resetreq`	Conduit	An optional local reset ports which appear after you enable **Add Reset Request Interface** parameter. The signal consists of an input `resetreq` signal and an output `ack` signal that trigger the Nios V processor to reset without affecting other components in a Nios V processor system. • You can request a reset to the Nios V processor core by asserting the `resetreq` signal. • The `resetreq` signal must remain asserted until the processor asserts `ack` signal. Failure for the signal to remain asserted can cause the processor to be in a non-deterministic state. • Assertion of the `resetreq` signal in debug mode has no effect on the processor's state. • The Nios V processor responds that the reset is successful by asserting the `ack` signal. • After the processor is successfully reset, the assertion of the ack signal can happen multiple times periodically until the de-assertion of the `resetreq` signal.

Figure 3. Nios V/g Processor Reset Network



2.3.4. Control and Status Registers

Nios V/m processor's Control and Status Registers (CSR) is both readable and writable. Nios V/m updates the CSR during the E-stage of the pipeline. If a memory or multicycle instruction is pending in M-stage, CSR write does not take place until M-stage completes this instruction. The application's write to CSR is processed by the core after M-stage completes, only if M-stage completes without an exception. If an exception is generated during M-stage, all CSR and other pending instructions are flushed and exception handle takes over.

2.3.5. Exception Controller

The Nios V/m processor architecture provides a simple exception controller to handle all exception types. Each exception, including internal hardware interrupts, causes the processor to transfer execution to an exception address. An exception handler at this address determines the cause of the exception and executes an appropriate exception routine.

You can set the exception address in the **Nios V Processor Board Support Package Editor ➤ BSP Linker Script**. Nios V/m processor stores the address in machine trap handler base address (`mtvec`) CSR register.

All exceptions are precise. The processor completes all instructions that precede the faulting instruction and does not start the execution of instructions that follow after the faulting instruction.

Table 8. Exceptions

Exception	Description
Instruction Address Misaligned	The core pipeline logic in F-stage detects the exception. This exception is flagged if the core fetched a program counter that is not aligned to a 32-bit word boundary.
Instruction Access Fault	The instruction read response signal detects this exception.
Illegal Instruction	The instruction decoder in the D-stage flags this exception if an instruction word contains encoding for an unimplemented or undefined instruction. The control logic for the CSR read and write flags this exception in the E-stage if a CSR instruction accesses a CSR that is not implemented or undefined.
Breakpoint	The instruction decoder flags the software breakpoint exception EBREAK in the D-stage.
Load Address Misaligned	The core for the load/store unit in the M-stage detects the misalignment. This exception is flagged if the data address is not aligned to the size of the data access.
Store Address Misaligned	
Load Access Fault	The core for the data read and write response signal detects the exception.
Store Access Fault	
Env call from M-mode	The instruction decoder in the D-stage detects the instruction.

2.3.6. Interrupt Controller

The Nios V/m processor implementation supports the following interrupts:

- Platform interrupts with 16 level-sensitive interrupt request (IRQ) inputs.
- Internally-generated Timer and Software interrupt. You can access the timer interrupt register using the Timer and Software interrupt module interface by connecting to the data bus.

During an interrupt, the core writes the program counter of the attached instruction into the machine exception program counter (`mepc`) register. An interrupt is usually attached to the instruction in E-stage or in the preceding F-stage or D-stage pipeline. Because the M-stage initiates the general-purpose register, the core is not capable of retracting a memory instruction in the M-stage. If an instruction in M-stage flags an exception while an interrupt is pending and ready to be serviced, the core fetches and executes the exception instruction. If a memory or multicycle instruction is pending in the M-stage, for example, the core is waiting for the response, the core does not flag an interrupt until it receives a response for that instruction. Pending interrupts are flagged by their corresponding bits in Machine Interrupt-Pending (`mip`) register.

An interrupt is taken only when Machine Status Register (`mstatus`) bit 3 is asserted and bits corresponding to its pending interrupt in Machine Interrupt-pending (`mip`) register is asserted.

Table 9. Interrupt Control and Status Registers/Bits

Register	Status Registers/Bits	Description
`mstatus`	`mstatus[3]`/Machine Interrupt-Enable (MIE) field	Global interrupt-enable bit for machine mode
`mie`	`mie[31:16]`/Platform interrupt-enable field	Platform interrupt-enable bit for 16 hardware interrupts
	`mie[7]`/Machine Timer Interrupt-Enable (MTIE) field	Timer interrupt-enable bit for machine mode
	`mie[3]`/Machine Software Interrupt-enable (MSIE) field	Software interrupt-enable bit for machine mode
`mip`	`mip[31:16]`/Platform interrupt-pending field	Platform interrupt-pending bit for 16 hardware interrupts
	`mip[7]`/Machine Timer Interrupt-Pending (MTIP) field	Timer interrupt-pending bit for machine mode
	`mip[3]`/Machine Software Interrupt-Pending (MSIP) field	Software interrupt-pending bit for machine mode

2.3.6.1. Timer and Software Interrupt Module

The timer and software interrupt hosts the following registers:

- Machine Time (`mtime`) and Machine Time Compare (`mtimecmp`) registers for timer interrupt.
- Machine Software Interrupt-pending (`msip`) field for the software interrupt.

The value of `mtime` increments after every clock cycle. When the value of `mtime` is greater or equal to the value of `mtimecmp`, the timer posts the interrupt.

2.3.7. Memory and I/O Organization

You can configure the Nios V/m processor systems. Consequently, the memory and I/O organization varies from system to system. A Nios V/m processor core uses one or more of the following ports to provide access to memory and I/O:

- Instruction manager port: An Arm* Advanced Microcontroller Bus Architecture (AMBA*) 4 AXI Memory-Mapped manager port that connects to instruction memory via system interconnect fabric.
- Data manager port: An AMBA 4 AXI Memory-Mapped manager port that connects to data memory and peripherals via the system interconnect fabric.

Nios V/m Processor Core Memory Mapped I/O Access: Both data memory and peripherals are mapped into the address space of the data manager port. Nios V/m processor core uses little-endian byte ordering. Words and half-words are stored in memory with the more-significant bytes at higher addresses. The Nios V/m processor core does not specify anything about the existence of memory and peripherals. The quantity, type, and connection of memory and peripherals are system dependent.

2.3.7.1. Instruction and Data Buses

2.3.7.1.1. Instruction Manager Port

Nios V/m processor instruction bus is implemented as a 32-bit AMBA 4 AXI manager port.

The instruction manager port:

- Performs a single function: it fetches instructions to be executed by the processor.
- Does not perform any write operations.
- Can issue successive read requests before data return from prior requests.
- Can prefetch sequential instructions.
- Always retrieves 32-bit of data. Every instruction fetch returns a full instruction word, regardless of the width of the target memory. The widths of memory in the Nios V/m processor system is not applicable to the programs. Instruction address is always aligned to a 32-bit word boundary.

Table 10. Instruction Interface Signals

Interface	Signal	Role	Direction
Write Address Channel	awaddr	Unused	Output
	awprot	Unused	Output
	awsize	Unused	Output
	awready	Unused	Input
Write Data Channel	wvalid	Unused	Output
	wdata	Unused	Output
	wstrb	Unused	Output
	wlast	Unused	Output
	wready	Unused	Input
Write Response Channel	bvalid	Unused	Input
	bres	Unused	Input
	bready	Unused	Output
Read Address Channel	araddr	Instruction Address (Program Counter)	Output
	arprot	Unused- tied off to constant value	Output
	arvalid	Instruction request valid	Output
	arsize	Constant 2- 4 bytes	Output
	arready	From subordinate/interconnect	Input
Read Data Channel	rdata	Instruction	Input
	rvalid	Instruction valid	Input
	rresp	Instruction response: Non-zero value denotes instruction access fault exception	Input
	rready	Constant 1	Output

2.3.7.1.2. Data Manager Port

The Nios V/m processor data bus is implemented as a 32-bit AMBA 4 AXI manager port. The data manager port performs two functions:

- Read data from memory or a peripheral when the processor executes a load instruction.
- Write data to memory or a peripheral when the processor executes a store instruction.

`axsize` signal value indicates the load/store instruction size- byte (LB/SB), halfword (LH/SH) or word (LW/SW). Address on `axaddr` signal is always aligned to size of the transfer. For store instructions, respective writes strobe bits are asserted to indicate bytes being written.

Nios V/m processor core does not support speculative issue of load/store instruction. Hence, a core can issue only one load or store instruction and waits until the issued instruction is complete.

Table 11. Data Interface Signals

Interface	Signal	Role	Direction
Write Address Channel	`awaddr`	Store address	Output
	`awprot`	Undefined- constant value	Output
	`awvalid`	Store valid	Output
	`awsize`	Store size- SB, SH, SW	Output
	`awready`	From memory/interconnect	Input
Write Data Channel	`wvalid`	Store valid	Output
	`wdata`	Store data	Output
	`wstrb`	Byte position in word	Output
	`wlast`	Constant 1	Output
	`wready`	From memory/interconnect	Input
Write Response Channel	`bvalid`	Store done	Input
	`bresp [1:0]`	Store done status: Non-zero value denotes store access fault exception.	Input
	`bready`	Constant 1	Output
Read Address Channel	`araddr`	Load Address	Output
	`arprot`	Undefined- constant value	Output
	`arvalid`	Load Valid	Output
	`arsize`	Load size: LB, LH, LW	Output
	`arready`	From subordinate/interconnect	Input
Read Data Channel	`rdata`	Read data	Input

continued...

Interface	Signal	Role	Direction
	rvalid	Read data valid	Input
	rresp	Load response status: Non-zero value denotes load access fault exception.	Input
	rready	Constant 1	Output

2.3.7.2. Address Map

The address map for memories and peripherals in a Nios V/m processor system is design dependent. The following addresses are part of the processor:

1. Reset Address
2. Debug Exception Address
3. Exception Address
4. Timer and Software Interrupt Address

You can specify the Reset Address and Debug Exception Address in Platform Designer during system configuration. You can modify the Exception Address stored in the `mvtec` register. `mvtime` and `mtimecmp` register controls the timer interrupt. The `msip` register bit controls the software interrupt.

2.3.8. RISC-V based Debug Module

The Nios V/m processor architecture supports a RISC-V based debug module that provides on-chip emulation features to control the processor remotely from a host PC. PC-based software debugging tools communicate with the debug module and provide facilities, such as the following features:

- Reset Nios V processor core and timer module
- Download programs to memory
- Start and stop execution
- Set software breakpoints and watchpoints
- Analyze registers and memory

2.3.8.1. Debug Mode

You can enter the Debug Mode, as specified in the RISC-V architecture specification, in the following ways:

1. Halt from Debug Module
2. Software breakpoints
3. Trigger

Upon entering Debug Mode, Nios V processor completes the instruction in W-stage. By the order of priority, instruction in M-stage, E-stage, D-stage or F-stage takes the interrupt.

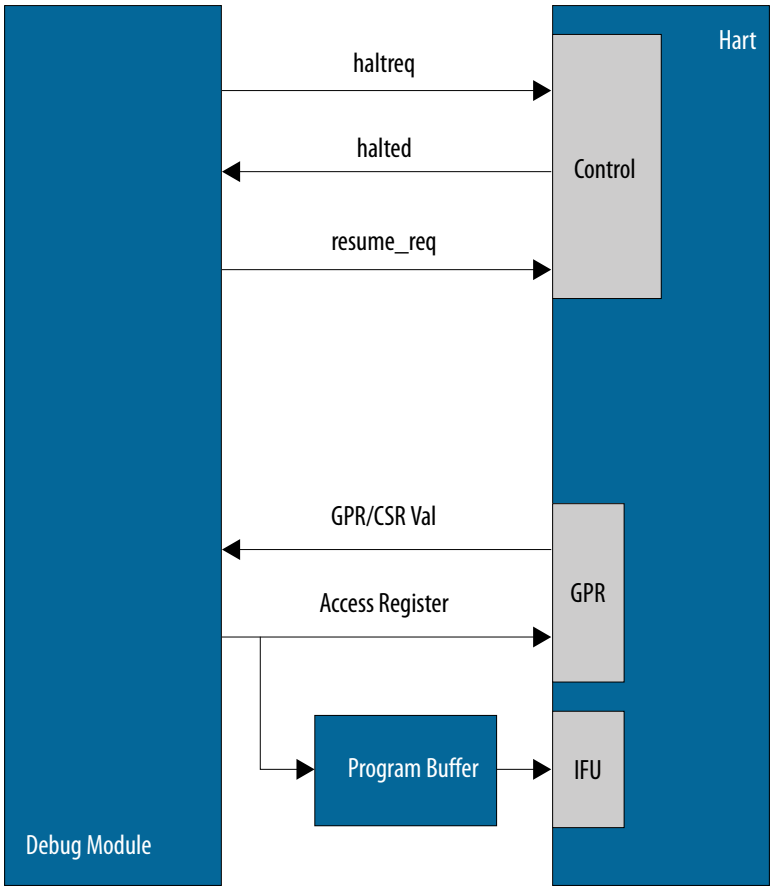
- When there is a valid instruction, Program Counter writes to the Debug Program Counter, `.dpc`.
- If the instruction in M-stage is not valid, then instruction in E-stage takes the interrupt and so on and so forth.
- If there is no valid instruction in the pipeline, the Program Counter for the next instruction writes to the Debug Program Counter, `.dpc`.

Note: For branches, the next Program Counter depends on whether a branch was taken or not taken, and whether the branch prediction (if any) was correct.

Debug Module selects Hardware Thread (Hart); which can be in one of the following states:

1. Non-existent: Debug Module probes a hart which does not exist.
2. Unavailable: Reset or temporary shutdown.
3. Running: Normal operation outside of debug.
4. Halted: Hart is said to be halted when it is in debug mode.

Figure 4. Debug Module Block Diagram



2.3.8.2. Halt from Debug Module

Debugger can write to the `haltreq` bit in Debug Module Control (`dmcontrol`) register, which places the Nios V processor in debug mode after some handshake. The assertion of `haltreq` bit sends an asynchronous interrupt to the processor core logic.

2.3.8.3. Trigger

The Nios V processor core supports one address or data match trigger. The trigger registers are accessible using RISC-V `csr` opcodes or abstract debug commands. The firing of trigger can either enter the Debug Mode or raise a breakpoint exception, which depends on the trigger registers.

Table 12. Trigger Registers Implemented in Nios V Processor

Name	Registers	Description
`tselect`	Trigger Select	Nios V processor supports one trigger, therefore the processor selects Trigger 0 at default (value set at 0).
`tdata1`	Trigger Data 1	The value of `type` field is 2 to represent Trigger 0 as an address or data match trigger. Write behavior to `tdata` registers depends on the `dmode` field. The remaining bits acts as `mcontrol`.
`mcontrol`	Match Control	Controls address and data trigger implementation according to `action`, `m`, `execute`, `store` and `load` fields. The Nios V processor does not implement other fields.
`tdata2`	Trigger Data 2	Holds trigger-specific data (virtual address, instruction opcode, data stored or loaded).
`tinfo`	Trigger Info	The value of info field is 4 at default to signify `tdata1.type` is 2. This implies selected trigger is an address or data match trigger.

Based on the bit field setting in `mcontrol` register, Nios V processor can implement different trigger types after the selected trigger matches the `tdata2` register.

Table 13. Nios V Processor Triggers Condition and Firing Time

Triggers	Condition	Firing Time	Exception Program Counter
Instruction Address Trigger	Program Counter matches `tdata2`	Before executing the instruction	The processor sets the Machine Exception Program Counter (`mepc`) to the instruction address (PC).
Instruction Opcode Trigger	Instruction opcode matches `tdata2`	Before executing the instruction	
Store Address Trigger	Store address matches `tdata2`	After executing the store instruction	The processor sets the `mepc` to the next instruction address (PC + 4).
Store Data Trigger	Store data matches `tdata2`	After executing the store instruction	
Load Address Trigger	Load address matches `tdata2`	After executing the load instruction	
Load Data Trigger	Load data matches `tdata2`	After executing the load instruction	

Table 14. Address or Data Match Trigger Type

Trigger	mcontrol bit fields			
	select	execute	store	load
Instruction Address	0	1	0	0
Instruction Opcode	1	1	0	0
Store Address	0	0	1	0
Store Data	1	0	1	0
Load Address	0	0	0	1
Load Data	1	0	0	1

Note: The trigger is disabled in the following situations:

- The Nios V processor is in debug mode.
- The Nios V processor is in machine mode, while `mcontrol.m` or `mcontrol.type` is 0.

2.3.8.4. Abstract Commands in Debug Mode

Nios V/m processor implements Access Register abstract command. The Access Register command allows read-write access to the processor registers including GPRs, CSRs, FP registers and Program Counter. The Access Register also allows program execution from program buffer. The debugger executes Access Register commands by writing into Abstract Command (`command`) register using the Access Register command encoding.

Table 15. Access Register Command Encoding

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
cmdtype								0	aarsize			aarpostincrement	postexec	transfer	write
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
regno															

Table 16. Fields Descriptions

Field	Role
`cmdtype`	Determine command type 0 : Indicates Access Register command.
`aarsize`	Specifies size of register access 2: Access the lowest 32 bit of register 3: Access the lowest 64 bit of register 4: Access the lowest 128 bit of register
`aarpostincrement`	0: No effect 1: regno is incremented after successful register access
`postexec`	0: No effect 1: Execute program in program buffer

continued...

Field	Role
transfer	Acts in conjunction with write field. 0: Ignore value in write field 1: Execute operation specified by write field.
write	0: Copy data from register 1: Copy data to register
regno	Register address to be accessed.

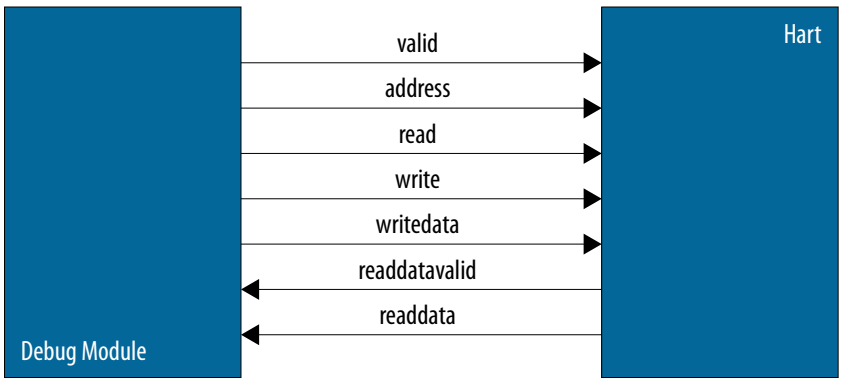
Note: The Nios V/m processor does not support abstract commands when hardware thread is not halted.

Table 17. Software Response to Unimplemented Commands

Field	State	
cmderr[10:8]	0	No error
	1	Busy
	2	Command not supported
	3	Exception - from program buffer instruction
	4	Command not executed because hart unavailable, or not in correct state to execute command.
	5	Abstract command failed due to bus error
	6	RSVD
	7	Command failed for other reasons.

Debug Module has the Abstract Control and Status CSR which includes the `cmderr` field. The `cmderr` field represents the current state of abstract command being executed. If the `cmderr` field is non-zero, writes to the `command` register are ignored. To clear the `cmderr` field, write 1 for every bit in the field.

Figure 5. Debug Module Interface Signals



Avalon memory-mapped interface implement the Register Access using request/response bus with Debug Module being the initiator and core being the responder. Address bus carries the register ID.

2.3.8.5. Hardware/Software Interface

Debug Module read or write to Debug Module registers, which initiate the interaction with the debugger. Each register has a fixed address as specified in the RISC-V Debug Support specification. Debugger can determine the register implementation status by write or read from the Debug Module registers. Unimplemented registers return 0 when read.

Debugger can check the status of the system by reading Debug Module Status (`dmstatus`) register. Debug Module Status register is read-only and provides status of the Debug Module and the selected harts.

You can access a specific hart by writing to the `hartsel` field in `dmcontrol`. Other fields in `dmcontrol` specify the action a debugger can take.

Halt Summary 0 (`haltsum0`) register reflects the status of a hart (halted/not halted). The LSB of this register can reflect whether hart is halted or not. Other bits is always 0. This is a read-only register of the debugger.

Related Information

RISC-V Debug Release
More information about the conceptual view of the states, refer to Section : Overview of States, Figure: Run/Halt Debug State Machine for single hart systems.

2.4. Programming Model

2.4.1. Privilege Levels

The privilege levels in Nios V/m processor are designed based on the RISC-V architecture specification. The privilege levels available are Machine Mode (M-mode) and Debug Mode (D-mode).

Related Information

The RISC-V Instruction Set Manual Volume II: Privileged Architecture Version 1.11

2.4.1.1. Machine Mode (M-mode)

The default operating mode is Machine mode (M-mode). The core boots up into M-mode after power-on reset by default. The M-mode provides low-level access to the machine implementation and all CSRs.

2.4.1.2. Debug Mode (D-mode)

The Debug mode (D-mode) is an additional privilege level to support off-chip debugging and manufacturing test.

The core can enter D-mode using any of the following methods:

- After reset when bit is set in debug CSR.
- Using debug interrupt.
- Using EBREAK instruction when bit is set in debug CSR.
- Using single step.

2.4.2. Control and Status Registers (CSR) Mapping

Control and status registers report the status and change the behavior of the processor. Since the processor core only supports M-mode and D-mode, Nios V/m processor implements the CSRs supported by these two modes.

Table 18. Control and Status Registers List

Number	Privilege	Name	Description
Machine Information Register			
0xF11	MRO	mvendorid	Vendor ID. Refer to Vendor ID Register Fields.
0xF12	MRO	marchid	Architecture ID. Refer to Architecture ID Register Fields.
0xF13	MRO	mimpid	Implementation ID. Refer to Implementation ID Register Fields.
0xF14	MRO	mhartid	Hardware thread ID. Refer to Hardware Thread ID Register Fields.
Machine Trap Setup			
0x300	MRW	mstatus	Machine status register. Refer to Machine Status Register Fields.
0x301	MRW	misa	ISA and extensions. Refer to Machine ISA Register Fields.
0x304	MRW	mie	Machine interrupt-enable register. Refer to Machine Interrupt-Enable Register Fields.
0x305	MRW	mtvec	Machine trap-handler base address. Refer to Machine Trap-Handler Base Address Register Fields.
Machine Trap Handling			
0x341	MRW	mepc	Machine exception program counter. Refer to Machine Exception Program Counter Register Fields.
0x342	MRW	mcause	Machine trap cause. Refer to Machine Trap Cause Register Fields.
0x343	MRW	mtval	Machine bad address or instruction. Refer to Machine Trap Value Register Fields.
0x344	MRW	mip	Machine interrupt pending. Refer to Machine Interrupt-Pending Register Fields.
Debug Mode Registers			
0x7B0	DRW	dcsr	Debug control and status register. Refer to Debug Control and Status Register Fields.
0x7B1	DRW	dpc	Debug Program Counter. Refer to Debug Program Counter Register Fields.
Trigger Registers			
0x7A0	MRW	tselect	Trigger select. Refer to Trigger Select Register Fields.
0x7A1	MRW	tdata1 (mcontrol)	Trigger data 1 (Match Control). Refer to Trigger Data 1 (Match Control) Register Fields
0x7A2	MRW	tdata2	Trigger data 2. Refer to Trigger Data 2 Register Fields.
0x7A4	MRO	tinfo	Trigger info. Refer to Trigger Info Register Fields.

Related Information

The RISC-V Instruction Set Manual Volume II: Privileged Architecture
More information about M-mode CSR and their respective fields.

2.4.2.1. Control and Status Register Field

The value in the each CSR registers determines the state of the Nios V/m processor. The field descriptions are based on the RISC-V specification.

Table 19. Vendor ID Register Fields

The `mvendorid` CSR is a 32-bit read-only register that provides the JEDEC manufacturer ID of the provider of the core.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bank															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bank									Offset						

Table 20. Architecture ID Register Fields

The `marchid` CSR is a 32-bit read-only register encoding the base microarchitecture of the hart.

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
Architecture ID														

Table 21. Implementation ID Register Fields

The `mimpid` CSR provides a unique encoding of the version of the processor implementation.

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
Implementation														

Table 22. Hardware Thread ID Register Fields

The `mhartid` CSR is a 32-bit read-only register that contains the integer ID of the hardware thread running the code

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
Hart ID														

Table 23. Machine Status Register Fields

The `mstatus` register is a 32-bit read-write register that keeps track of and controls the hart's current operating state.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SD	WPRI								TSR	TW	TVM	MXR	SUM	MPRV	XS[1:0]
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
XS[1:0]	FS[1:0]		MPP[1:0]		WPRI		SPP	MPIE	WPRI	SPIE	UPIE	MIE	WPRI	SIE	UIE

Table 24. Machine ISA Register Fields

The `misa` CSR is a read-write register reporting the ISA supported by the hart.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MXL[1:0]		WLRL				Extension[25:0]									
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Extension[25:0]															

Table 25. Machine Interrupt-Enable Register Fields

The `mie` register is a 32-bit read-write register that contains interrupt enable bits.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WPRI															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WPRI				MEIE	WPRI	SEIE	UEIE	MTIE	WPRI	STIE	UTIE	MSIE	WPRI	SSIE	USIE

Table 26. Machine Trap-Handler Base Address Register Fields

The `mtvec` register is a 32-bit read/write register that holds trap vector configuration, consisting of a vector base address (BASE) and a vector mode (MODE).

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Base[31:2]															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Base[31:2]														Mode	

Table 27. Machine Exception Program Counter Register Fields

The `mepc` register is a 32-bit read-write register that holds the addresses of the instruction that was interrupted or that encountered the exception when a trap is taken into M-mode.

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
mepc														

Table 28. Machine Trap Cause Register Fields

The `mcause` register is a 32-bit read-write register that hold the code indicating the event that caused the trap when a trap is taken into M-mode.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Interrupt	Exception code [30:16]														
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Exception code [15:0]															

Table 29. Machine Trap Value Register Fields

The `mtval` register is a 32-bit read-write register that is written with exception-specific information to assist software in handling the trap when a trap is taken into M-mode.

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
mtval														

Table 30. Machine Interrupt-Pending Register Fields

The `mip` register is a 32-bit read/write register containing information on pending interrupts.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WPRI															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WPRI				MEIP	WPRI	SEIP	UEIP	MTIP	WPRI	STIP	UTIP	MSIP	WPRI	SSIP	USIP

Table 31. Trigger Select Register Fields

The `tselect` register is a 32-bit read/write register that selects the current trigger is accessible by other trigger register.

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
tselect														

Table 32. Trigger Data 1 (Match Control) Register Fields

The `tdata1 (mcontrol)` register is a 32-bit read/write register containing information on the trigger type, tdata registers accessibility, and trigger implementation

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
type=2				dmode	maskmax						hit	select	timing	sizelo	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
action				chain	match				m	0	s	u	execute	store	load

Table 33. Trigger Data 2 Register Fields

The `tdata2` register is a 32-bit read/write register containing the trigger-specific data.

Bit Field														
31	30	29	28	27	26	25	…	6	5	4	3	2	1	0
tdata2														

Table 34. Trigger Info Register Fields

The `tinfo` register is a 32-bit read-only register containing information on each possible tdata1.type.

Bit Field													
31	30	29	…	18	17	16	15	14	13	…	2	1	0
0							info						

Table 35. Debug Control and Status Register Fields

The `dcsr` CSR is a 32-bit read-write register containing information and status during D-mode

Bit Field														
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
debugver				0											
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ebreakm	0	ebreaks	ebreaku	stepie	stopcount	stoptime	cause			0	mprven	nmip	step	prv	

Table 36. Debug Program Counter Register Fields

Upon entry to D-mode, `dpc` CSR is updated with the virtual address of the next instruction to be executed.

Bit Field														
31	30	29	28	27	26	25	…	6	5	4	3	2	1	0
dpc														

2.5. Core Implementation

2.5.1. Instruction Set Reference

The Nios V/m processor is based on the RV32IA specification, and there are 6 types of instruction formats. They are R-type, I-type, S-type, B-type, U-type, and J-type.

Table 37. Instruction Formats (R-type)

Bit Field (R-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
funct7							rs2					rs1			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs1	funct3			rd			opcode								

Table 38. Instruction Formats (I-type)

Bit Field (I-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
imm[11:0]												rs1			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs1	funct3			rd					opcode						

Table 39. Instruction Formats (S-type)

Bit Field (S-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
imm[11:5]							rs2					rs1			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs1	funct3			imm[4:0]					opcode						

Table 40. Instruction Formats (B-type)

Bit Field (B-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
imm[12]	imm[10:5]						rs2					rs1			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs1	funct3			imm[4:1]			imm[11]		opcode						

Table 41. Instruction Formats (U-type)

Bit Field (U-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
imm[31:16]															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
imm[15:12]				rd					opcode						

Table 42. Instruction Formats (J-type)

Bit Field (J-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
imm[20]	imm[10:1]										imm[11]	imm[19:16]			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
imm[15:12]				rd					opcode						

Related Information

The RISC-V Instruction Set Manual Volume I: Unprivileged ISA
More information about the instruction set of the RV32I Base Integer and the "A" standard extension.

3. Nios V/g Processor

The Nios V/g processor is a general-purpose core developed by Intel based on the RISC-V RV32IMAZicsr_Zicbom instruction set and supports the functional units described in this document.

Related Information

Overview on page 3

3.1. Processor Performance Benchmarks

Table 43. Nios V/g Processor Performance Benchmarks in Intel FPGA Devices for Intel Quartus Prime Pro Edition Software

FPGA Used	f_{MAX} (MHz)	Logic Size (ALM)	Architecture Performance	
			DMIPS/MHz Ratio	CoreMark/MHz Ratio
Intel Cyclone 10	233.71	2511	0.866	1.467
Intel Arria 10	240.79	2504		
Intel Stratix 10	272.70	2556		
Intel Agilex 7	321.50	2480		

Table 44. Benchmark Parameters for Intel Quartus Prime Pro Edition Software

Parameter		Settings/Description
Intel Quartus Prime seed		Maximum performance result are based on 10 seed sweep from Intel Quartus Prime Pro Edition software version 23.1.
Device speed grade		Fastest speed grade from each Intel FPGA device family.
Defined peripherals		• Nios V/g processor core (without debug module, 4 KB instruction cache, and 4 KB data cache). • 128 KB on-chip memory for the instruction and data bus. • JTAG UART Intel FPGA IP.
Toolchain	Version	• riscv32-unknown-elf-gcc (GCC) version 12.1.0 • CMake Version: 3.23.2
	Compiler configuration	• Compiler flags: `-03` • Assembler options: `-Wa -gdwarf2` • Compile options: `-Wall -Wformat-security -march=rv32ima_zicbom -mabi=ilp32`

Intel uses the same Intel Quartus Prime design example for maximum performance benchmark(fMAX) and logic size benchmarks. However, the compiler settings are different for each benchmarks:

- fMAX benchmark: `superior_performance_optimized_placement_effort`
- Logic size benchmark: `area_aggressive`

Note: Results may vary depending on the version of the Intel Quartus Prime software, the version of the Nios V processor, compiler version, target device and the configuration of the processor. Additionally, any changes to the system logic design might change the performance and LE usage. All results are generated from design built with Platform Designer.

3.2. Processor Pipeline

The Nios V/g processor employs a five-stage pipeline.

Table 45. Processor Pipeline Stages

Stage	Denotation	Function
F	Instruction fetch	• PC+4 calculation • Next instruction fetch • Pre-decode for register file read
D	Instruction decode	• Decode the instruction • Register file read data available • Hazard resolution and data forwarding
E	Instruction execute	• ALU operations • Memory address calculation • Branch resolution • CSR read/write
M	Memory	• Memory and multicycle operations • Register file write • Branch redirection
W	Write back	• Facilitates data dependency resolution by providing general-purpose register value.

The Nios V/g processor implements the general-purpose register file using the M20K memory blocks. The processor takes one processing cycle to read from an M20K location. Therefore, the F-stage initiates register file reads so general-purpose register values are available in D-stage.

Writing to the M20K location takes two processing cycles. Therefore, the M-stage initiates writes to a general-purpose register. If there is a dependency to resolve, the M-stage carries forward the value to the W-stage.

The core resolves data dependencies in the D-stage. Operands can move from register file read or E-stage, M-stage, or W-stage.

Reasons for the pipeline stalling:

- Data dependency—if the source operand is not available in D-stage, instruction in D-stage and F-stage stalls until the operand becomes available. The scenario can happen if destination general-purpose register of load or multicycle instruction in E-stage or M-stage is the source for instruction in D-stage.
- Resource stall—if a memory operation or multicycle is pending in M-stage, the instructions in preceding stages stalls until M-stage completes the instruction.

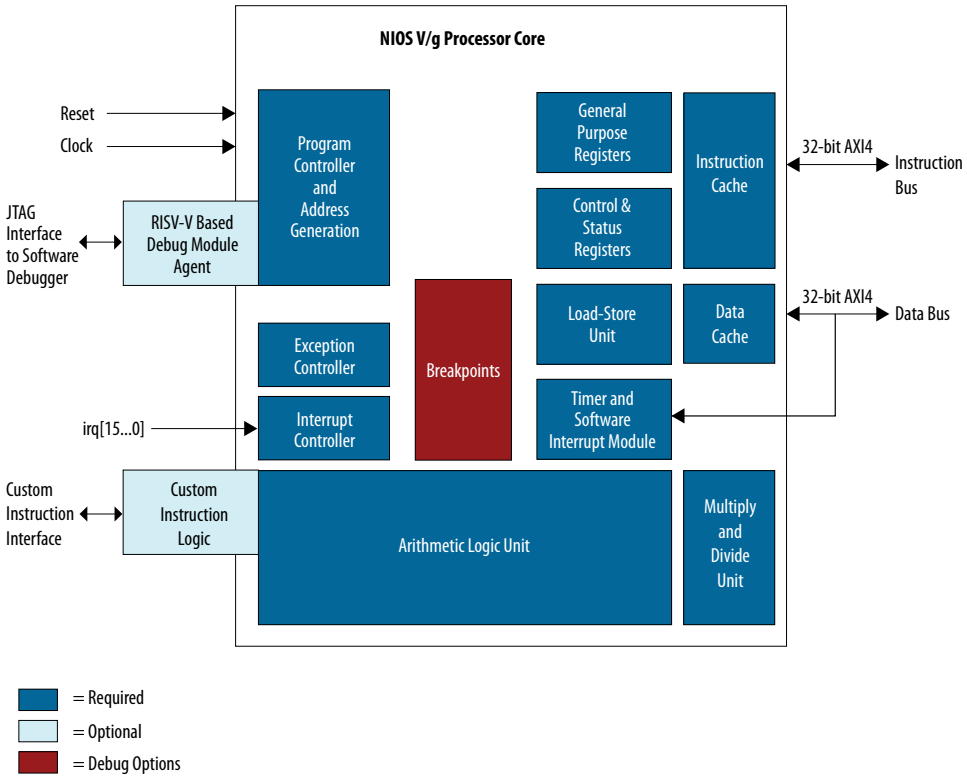
3.3. Processor Architecture

The Nios V/g processor architecture describes an instruction-set architecture (ISA). The ISA in turn necessitates a set of functional units that implement the instructions.

The Nios V/g processor architecture defines the following functional units:

- General-purpose register file
- Arithmetic logic unit (ALU)
- Multiply and divide units
- Custom instruction logic
- Control and status registers (CSR)
- Exception controller
- Interrupt controller
- Instruction bus
- Data bus
- Instruction cache
- Data cache
- RISC-V based debug module

Figure 6. Nios V/g Processor Core Block Diagram



3.3.1. General-Purpose Register File

Nios V/g processor implementation supports a flat register file. The register file contains thirty-two 32-bit general-purpose integer registers. Nios V/g processor implements the general-purpose register using M20K memories, which do not support two read ports. Hence, Nios V/g processor duplicates the register files so that two different source registers for an instruction are available in a single cycle. After performing ALU operations, the processor core writes the same result to the destination register in both memories.

3.3.2. Arithmetic Logic Unit

The arithmetic logic unit (ALU) operates on data stored in general-purpose registers. ALU operations take one or two inputs from registers and store the result back into the register.

Table 46. Fundamental Data Operations of the ALU

Category	Description
Arithmetic	Addition and subtraction on signed and unsigned operands.
Relational	Equal, not-equal, greater-than-or-equal, and less-than relational operations (==, != >=, <).
Logical	AND, OR, NOR, and XOR logical operations.
Shift	Logical and arithmetic shift operations.

For load and store instructions, the Nios V/g processor uses the ALU to calculate the memory address. For conditional control transfer instructions, Nios V/g processor uses the relational operations in the ALU to determine if the processor takes or leaves the branch..

3.3.3. Multipy and Divide Units

The multiply and divide units take two inputs from registers, compute, and store the result to the register.

3.3.4. Custom Instruction

The Nios V/g processor architecture supports user-defined custom instructions. The Nios V/g ALU connects directly to custom instruction logic, enabling you to implement operations in hardware that are accessed and used in the same way as native instructions.

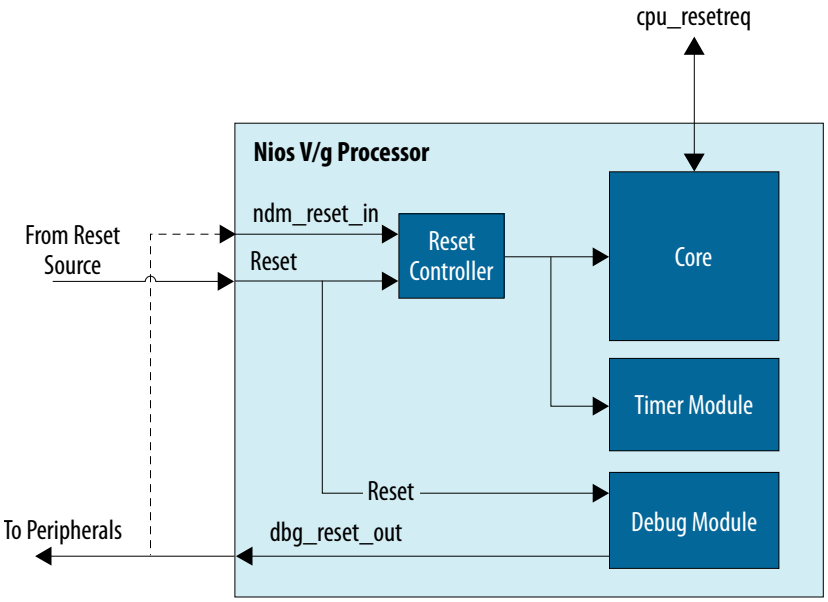
Related Information

AN 977: Nios V Processor Custom Instruction

3.3.5. Reset and Debug Signals

Interface	Type	Description
`reset`	Reset	A global hardware reset input signal that forces the Nios V processor to reset immediately.
`dbg_reset_out`	Reset	An optional reset output signal which appear after you enable both **Enable Debug** and **Enable Reset from Debug Module** parameters. • This reset output signal is triggered by the JTAG debugger or `niosv-download -r` command. • You can connect this reset output signal to the following input signals: — To the `ndm_reset_in` input to reset the core and the timer module. — To the reset input signal of other components as needed.
`ndm_reset_in`	Reset	An optional reset input signal which appear after you enable both **Enable Debug** and **Enable Reset from Debug Module** parameters. • You can use this signal to trigger reset controller to reset the core and the timer module. • The reset controller synchronizes the reset (hard reset) and `ndm_reset_in` signals.
`cpu_resetreq`	Conduit	An optional local reset ports which appear after you enable **Add Reset Request Interface** parameter. The signal consists of an input `resetreq` signal and an output `ack` signal that trigger the Nios V processor to reset without affecting other components in a Nios V processor system. • You can request a reset to the Nios V processor core by asserting the `resetreq` signal. • The `resetreq` signal must remain asserted until the processor asserts `ack` signal. Failure for the signal to remain asserted can cause the processor to be in a non-deterministic state. • Assertion of the `resetreq` signal in debug mode has no effect on the processor's state. • The Nios V processor responds that the reset is successful by asserting the `ack` signal. • After the processor is successfully reset, the assertion of the ack signal can happen multiple times periodically until the de-assertion of the `resetreq` signal.

Figure 7. Nios V/g Processor Reset Network



3.3.6. Control and Status Registers

Nios V/g processor's Control and Status Registers (CSR) is both readable and writable. Nios V/g processor updates the CSR during the E-stage of the pipeline. If a memory or multicycle instruction is pending in M-stage, CSR write does not take place until M-stage completes this instruction. The application's write to CSR is processed by the core after M-stage completes, only if M-stage completes without an exception. If an exception is generated during M-stage, all CSR and other pending instructions are flushed and exception handle takes over.

3.3.7. Exception Controller

The Nios V/g processor architecture provides a simple exception controller to handle all exception types. Each exception, including internal hardware interrupts, causes the processor to transfer execution to an exception address. An exception handler at this address determines the cause of the exception and executes an appropriate exception routine.

You can set the exception address in the **Nios V Processor Board Support Package Editor ➤ BSP Linker Script**. Nios V/g processor stores the address in machine trap handler base address (`mtvec`) CSR register.

All exceptions are precise. The processor completes all instructions that precede the faulting instruction and does not start the execution of instructions that follow after the faulting instruction.

Table 47. Exceptions

Exception	Description
Instruction Address Misaligned	The core pipeline logic in F-stage detects the exception. This exception is flagged if the core fetched a program counter that is not aligned to a 32-bit word boundary.
Instruction Access Fault	The instruction read response signal detects this exception.
Illegal Instruction	The instruction decoder in the D-stage flags this exception if an instruction word contains encoding for an unimplemented or undefined instruction. The control logic for the CSR read and write flags this exception in the E-stage if a CSR instruction accesses a CSR that is not implemented or undefined.
Breakpoint	The instruction decoder flags the software breakpoint exception EBREAK in the D-stage.
Load Address Misaligned	The core for the load/store unit in the M-stage detects the misalignment. This exception is flagged if the data address is not aligned to the size of the data access.
Store Address Misaligned	
Load Access Fault	The core for the data read and write response signal detects the exception.
Store Access Fault	
Env call from M-mode	The instruction decoder in the D-stage detects the instruction.

3.3.8. Interrupt Controller

The Nios V/g processor implementation supports the following interrupts:

- Platform interrupts with 16 level-sensitive interrupt request (IRQ) inputs.
- Internally-generated Timer and Software interrupt. You can access the timer interrupt register using the Timer and Software interrupt module interface by connecting to the data bus.

During an interrupt, the core writes the program counter of the attached instruction into the machine exception program counter (`mepc`) register. An interrupt is usually attached to the instruction in E-stage or in the preceding F-stage or D-stage pipeline. Because the M-stage initiates the general-purpose register, the core is not capable of retracting a memory instruction in the M-stage. If an instruction in M-stage flags an exception while an interrupt is pending and ready to be serviced, the core fetches and executes the exception instruction. If a memory or multicycle instruction is pending in the M-stage, for example, the core is waiting for the response, the core does not flag an interrupt until it receives a response for that instruction. Pending interrupts are flagged by their corresponding bits in Machine Interrupt-Pending (`mip`) register.

An interrupt is taken only when Machine Status Register (`mstatus`) bit 3 is asserted and bits corresponding to its pending interrupt in Machine Interrupt-pending (`mip`) register is asserted.

Table 48. Interrupt Control and Status Registers/Bits

Register	Status Registers/Bits	Description
mstatus	mstatus[3]/Machine Interrupt-Enable (MIE) field	Global interrupt-enable bit for machine mode
mie	mie[31:16]/Platform interrupt-enable field	Platform interrupt-enable bit for 16 hardware interrupts
	mie[7]/Machine Timer Interrupt-Enable (MTIE) field	Timer interrupt-enable bit for machine mode
	mie[3]/Machine Software Interrupt-enable (MSIE) field	Software interrupt-enable bit for machine mode
mip	mip[31:16]/Platform interrupt-pending field	Platform interrupt-pending bit for 16 hardware interrupts
	mip[7]/Machine Timer Interrupt-Pending (MTIP) field	Timer interrupt-pending bit for machine mode
	mip[3]/Machine Software Interrupt-Pending (MSIP) field	Software interrupt-pending bit for machine mode

3.3.8.1. Timer and Software Interrupt Module

The timer and software interrupt hosts the following registers:

- Machine Time (`mtime`) and Machine Time Compare (`mtimecmp`) registers for timer interrupt.
- Machine Software Interrupt-pending (`msip`) field for the software interrupt.

The value of `mtime` increments after every clock cycle. When the value of `mtime` is greater or equal to the value of `mtimecmp`, the timer posts the interrupt.

3.3.9. Memory and I/O Organization

You can configure the Nios V/g processor systems. Consequently, the memory and I/O organization varies from system to system. A Nios V/g processor core uses one or more of the following ports to provide access to memory and I/O:

- Instruction manager port: An Arm Advanced Microcontroller Bus Architecture (AMBA) 4 AXI Memory-Mapped manager port that connects to instruction memory via system interconnect fabric.
- Data manager port: An AMBA 4 AXI Memory-Mapped manager port that connects to data memory and peripherals via the system interconnect fabric.
- Instruction Cache: Fast cache memory internal to the Nios V/g processor core.
- Data Cache: Fast cache memory internal to the Nios V/g processor core.

Nios V/g Processor Core Memory Mapped I/O Access: Both data memory and peripherals are mapped into the address space of the data manager port. Nios V/g processor core uses little-endian byte ordering. Words and half-words are stored in memory with the more-significant bytes at higher addresses. The Nios V/g processor core does not specify anything about the existence of memory and peripherals. The quantity, type, and connection of memory and peripherals are system dependent.

3.3.9.1. Instruction and Data Buses

3.3.9.1.1. Instruction Manager Port

Nios V/g processor instruction bus is implemented as a 32-bit AMBA 4 AXI manager port.

The instruction manager port:

- Performs a single function: it fetches instructions to be executed by the processor.
- Does not perform any write operations.
- Can issue successive read requests before data return from prior requests.
- Can prefetch sequential instructions.
- Always retrieves 32-bit of data. Every instruction fetch returns a full instruction word, regardless of the width of the target memory. The widths of memory in the Nios V/g processor system is not applicable to the programs. Instruction address is always aligned to a 32-bit word boundary.

Table 49. Instruction Interface Signals

Interface	Signal	Role	Direction
Write Address Channel	awaddr	Unused	Output
	awprot	Unused	Output
	awsize	Unused	Output
	awready	Unused	Input
	awvalid	Unused	Output
	awlen	Unused	Output
Write Data Channel	wvalid	Unused	Output
	wdata	Unused	Output
	wstrb	Unused	Output
	wlast	Unused	Output
	wready	Unused	Input
Write Response Channel	bvalid	Unused	Input
	bresp	Unused	Input
	bready	Unused	Output
Read Address Channel	araddr	Instruction Address (Program Counter)	Output
	arprot	Unused- tied off to constant value	Output
	arvalid	Instruction request valid	Output
	arsize	Constant 2- 4 bytes	Output
	arready	From subordinate/interconnect	Input
	awlen	Read burst length • 0 for peripheral region access • 7 for cacheable region access	Output
Read Data Channel	rdata	Instruction	Input
	rvalid	Instruction valid	Input

continued...

Interface	Signal	Role	Direction
	rresp	Instruction response: Non-zero value denotes instruction access fault exception	Input
	rready	Constant 1	Output
	rlast	Last transfer in a read burst	Input

3.3.9.1.2. Data Manager Port

The Nios V/g processor data bus is implemented as a 32-bit AMBA 4 AXI manager port. The data manager port performs two functions:

- Read data from memory or a peripheral when the processor executes a load instruction.
- Write data to memory or a peripheral when the processor executes a store instruction.

axsize signal value indicates the load/store instruction size- byte (LB/SB), halfword (LH/SH) or word (LW/SW). Address on axaddr signal is always aligned to size of the transfer. For store instructions, respective writes strobe bits are asserted to indicate bytes being written.

Nios V/g processor core does not support speculative issue of load/store instruction. Hence, a core can issue only one load or store instruction and waits until the issued instruction is complete.

Table 50. Data Interface Signals

Interface	Signal	Role	Direction
Write Address Channel	awaddr	Store address	Output
	awprot	Undefined- constant value	Output
	awvalid	Store valid	Output
	awsize	Store size- SB, SH, SW	Output
	awready	From memory/interconnect	Input
	awlen	Write burst length • 0 for peripheral region access • 7 for cacheable region access	Output
Write Data Channel	wvalid	Store valid	Output
	wdata	Store data	Output
	wstrb	Byte position in word	Output
	wlast	Last transfer in a write burst	Output
	wready	From memory/interconnect	Input
Write Response Channel	bvalid	Store done	Input
	bresp [1:0]	Store done status: Non-zero value denotes store access fault exception.	Input
	bready	Constant 1	Output

continued...

Interface	Signal	Role	Direction
Read Address Channel	araddr	Load Address	Output
	arprot	Undefined- constant value	Output
	arvalid	Load Valid	Output
	arsize	Load size: LB, LH, LW	Output
	arready	From subordinate/interconnect	Input
	awlen	Read burst length • 0 for peripheral region access • 7 for cacheable region access	Output
Read Data Channel	rdata	Read data	Input
	rvalid	Read data valid	Input
	rresp	Load response status: Non-zero value denotes load access fault exception.	Input
	rready	Constant 1	Output
	rlast	Last transfer in a read burst	Input

3.3.9.2. Address Map

The address map for memories and peripherals in a Nios V/g processor system is design dependent. The following addresses are part of the processor:

1. Reset Address
2. Debug Exception Address
3. Peripheral Region Base Address
4. Exception Address
5. Timer and Software Interrupt Address

You can specify the Reset Address, Debug Exception Address and Peripheral Region Base Address in Platform Designer during system configuration. You can modify the Exception Address stored in the `mvtec` register. `mvtime` and `mtimecmp` register controls the timer interrupt. The `msip` register bit controls the software interrupt.

3.3.9.3. Cache Memory

The Nios V/g processor architecture supports cache memories on both the instruction manager port (instruction cache) and the data manager port (data cache). The cache memories can improve the average memory access time for Nios V/g processor systems that use slow off-chip memory such as SDRAM for programme and data storage.

The processor core connects the caches through a 32-bit AXI-4 interface, thus bursting is enabled. The instruction and data caches are always enabled at run-time, but software can bypass the data cache so that peripheral accesses do not return cached data. Software handles cache management and cache coherency. The Nios V/g instruction set provides instructions for cache management.

Table 51. Cache Byte Address Field

Bit Fields															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
tag															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
tag		tag/index				index					offset				

Related Information

RISC-V Base Cache Management Operation ISA Extension
More information on RISC-V Zicbom Cache Management Operation

3.3.9.3.1. Instruction Cache

The instruction cache memory has the following characteristics:

- Direct-mapped cache implementation
- 32 bytes (8 words) per cache line
- Configurable size of 1, 2, 4, 8, and 16 KBytes
- The instruction manager port reads an entire cache line at a time from memory, and issues one read per clock cycle.
- Critical word first

The instruction byte address size is 32-bit. The size of the tag and index field depends only on the size of the cache memory. The offset field is always five bits (i.e., a 32-byte line). The instruction fetch fence (`fence.i`) instruction flushes and invalidates all instruction cache lines.

Related Information

The RISC-V Instruction Set Manual Volume I: Unprivileged ISA
More information about the instruction set of the RV32I Base Integer and the "A" standard extension.

3.3.9.3.2. Data Cache

The data cache memory has the following characteristics:

- Direct-mapped cache implementation
- 32 bytes (8 words) per cache line
- Configurable size of 1, 2, 4, 8, and 16 KBytes
- The data manager port reads an entire cache line at a time from memory, and issues one read per clock cycle.
- Write-back
- Write-allocate (i.e., on a store instruction, a cache miss allocates the line for that address)

The data byte address size is 32 bit. The size of the `tag` and `index` field depends only on the size of the cache memory. The `offset` field is always five bits (i.e., a 32-byte line).

The Nios V/g processor instruction set provides cache block management instructions for the data cache.

Table 52. RISC-V Zicbom Cache Block Management

Instruction	Name	Operation	Encoding
`cbo.clean <rs1>`(1)(2)	Clean Data Cache Address	• Identifies the cache line with `tag` and `index` field. • If there is a cache hit, proceeds to the following operations: — Clears the cache line's dirty state. — Keeps the cache line's valid state. — If the cache line is valid and dirty, data is written back to the memory.	Refer to *RISC-V Base Cache Management Operation ISA Extension*.
`cbo.flush <rs1>`(1)(2)	Flush Data Cache Address	• Identifies the cache line with `tag` and `index` field. • If there is a cache hit, proceeds to the following operations: — Invalidates the cache line. — Writes the data back to the memory if the cache line is valid and dirty; otherwise, ignores the data.	
`cbo.inval <rs1>`(1)(2)	Invalidate Data Cache Address	• Identifies the cache line with `tag` and `index` field. • If there is a cache hit, proceeds to the following operations: — Invalidates the cache line. — Does not write the data back to the memory.	

Table 53. Custom Cache Block Management Instructions

Instruction	Operation	Encoding
`cbo.clean.ix <rs1>[`(3)	• Identifies the cache line with `index` field, • Clears the cache line's dirty state. • Keeps the cache line's valid state. • If the cache line is valid and dirty, data is written back to the memory.	Refer to Encoding for `cbo.clean.ix`
`cbo.flush.ix <rs1>`(3)	• Identifies the cache line with `index` • Invalidates the cache line. • If the cache line is valid and dirty, data is written back to the memory. • If the cache line is not dirty, data is not written back to the memory field,	Refer to Encoding for `cbo.flush.ix`
`cbo.inval.ix <rs1>[`(3)	• Identifies the cache line with index field, • Invalidates the cache line. • Data is not written back to the memory.	Refer to Encoding for `cbo.inval.ix`

(1) Source register 1 (`rs1`) holds the 32-bit cache line address (tag, index, and offset).

(2) If the specified cache line address is not found (cache miss), these instruction are implemented as `nop`, and does no changes to any cache lines.

(3) Source register 1 (`rs1`) holds the cache line index, which can be 5 to 9-bits depending on the cache size.

Table 54. Encoding for `cbo.clean.ix`

Bit Field				
31:20	19:15	14:12	11:7	6:0
imm	rs1	cbo	Reserved	misc-mem
000010000001	rs1	010	00000	0001111

Table 55. Encoding for `cbo.flush.ix`

Bit Field				
31:20	19:15	14:12	11:7	6:0
imm	rs1	cbo	Reserved	misc-mem
000010000010	rs1	010	00000	0001111

Table 56. Encoding for `cbo.inval.ix`

Bit Field				
31:20	19:15	14:12	11:7	6:0
imm	rs1	cbo	Reserved	misc-mem
000010000000	rs1	010	00000	0001111

3.3.9.3.3. Configurable Cache Memory

You can configure the size of the cache memory. The cache memory has no effect on programme functionality, but affect the speed with which the processor fetches instructions and reads/writes data.

3.3.9.3.4. Bypassing Cache (Peripheral Region)

The Nios V/g architecture has two peripheral regions for bypassing the caches. Nios V/g cores optionally support the peripheral region mechanism to indicate cacheability. In the Platform Designer, the peripheral region cache-ability mechanism allows you to specify a region of address space that is non-cacheable. The peripheral region is any integer power of 2 bytes, from a minimum of 64 kilobytes up to a maximum of 2 gigabytes, and must be located at a base address aligned to the size of the peripheral region. The peripheral region is available as long as an MMU is not present.

Note: Any accesses to the Nios V processor's debug module or timer module are non-cacheable.

Note: You must place the peripherals driven by the Nios V/g processor within a defined peripheral region to achieve cache bypass, which is required by a standard design implementation.

3.3.9.3.5. Using Cache Memory Effectively

The effectiveness of cache memory to improve performance is based on the following conditions:

- Regular memory is located off-chip and has a longer access time than on-chip memory.
- The largest, performance-critical instruction loop is smaller than the instruction cache.
- The largest block of performance-critical data is smaller than the data cache.

The optimal cache configuration is application-specific, but you can define a configuration that works for a variety of applications. Refer to the following examples:

- If a Nios V/g processor system only has fast on-chip memory and never accesses slow off-chip memory, an instruction or data cache is unlikely to boost the performance.
- If a program's critical loop is 2 KB but the instruction cache is 1 KB, an instruction cache does not improve execution speed. In this case, an instruction cache can actually degrade performance.

Mixing cached and uncached accesses to the same cache line can result in invalid data reads. For example, the following sequences of events causes cache incoherency.

1. Nios V processor writes data to cache, creating a dirty data cache line.
2. Nios V processor reads data from the same address, but bypassed the cache.

If it is necessary to mix cached and uncached data accesses, flush the corresponding line of data cache after completing the cached accesses and before performing the uncached accesses.

3.3.10. RISC-V based Debug Module

The Nios V/g processor architecture supports a RISC-V based debug module that provides on-chip emulation features to control the processor remotely from a host PC. PC-based software debugging tools communicate with the debug module and provide facilities, such as the following features:

- Reset Nios V processor core and timer module
- Download programs to memory
- Start and stop execution
- Set software breakpoints and watchpoints
- Analyze registers and memory

3.3.10.1. Debug Mode

You can enter the Debug Mode, as specified in the RISC-V architecture specification, in the following ways:

1. Halt from Debug Module
2. Software breakpoints
3. Trigger

Upon entering Debug Mode, Nios V processor completes the instruction in W-stage. By the order of priority, instruction in M-stage, E-stage, D-stage or F-stage takes the interrupt.

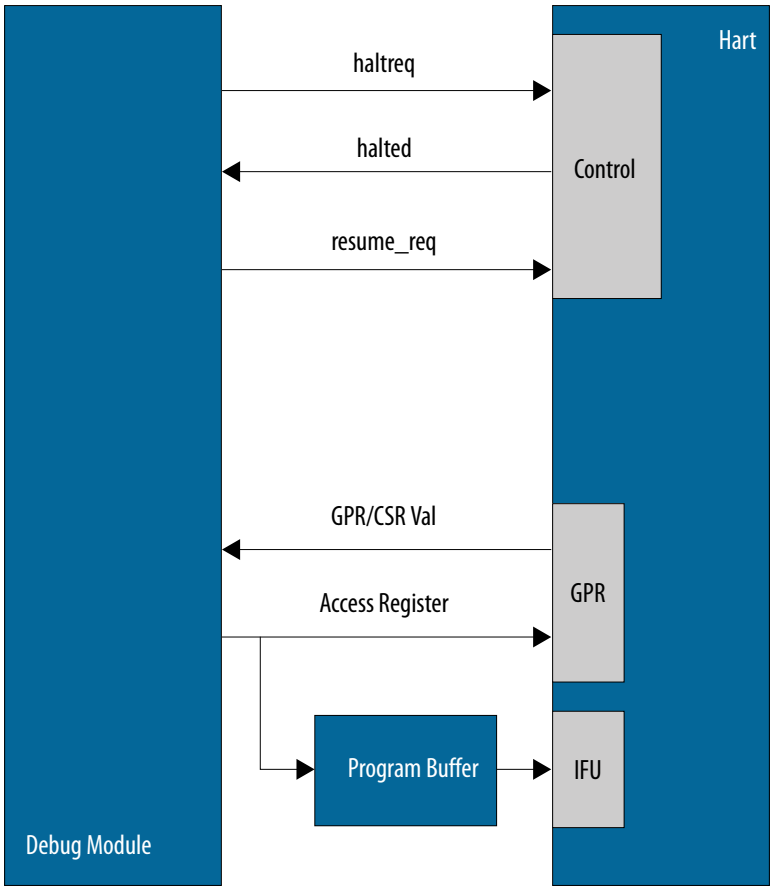
- When there is a valid instruction, Program Counter writes to the Debug Program Counter, `.dpc`.
- If the instruction in M-stage is not valid, then instruction in E-stage takes the interrupt and so on and so forth.
- If there is no valid instruction in the pipeline, the Program Counter for the next instruction writes to the Debug Program Counter, `.dpc`.

Note: For branches, the next Program Counter depends on whether a branch was taken or not taken, and whether the branch prediction (if any) was correct.

Debug Module selects Hardware Thread (Hart); which can be in one of the following states:

1. Non-existent: Debug Module probes a hart which does not exist.
2. Unavailable: Reset or temporary shutdown.
3. Running: Normal operation outside of debug.
4. Halted: Hart is said to be halted when it is in debug mode.

Figure 8. Debug Module Block Diagram



3.3.10.2. Halt from Debug Module

Debugger can write to the `haltreq` bit in Debug Module Control (`dmcontrol`) register, which places the Nios V processor in debug mode after some handshake. The assertion of `haltreq` bit sends an asynchronous interrupt to the processor core logic.

3.3.10.3. Trigger

The Nios V processor core supports one address or data match trigger. The trigger registers are accessible using RISC-V `csr` opcodes or abstract debug commands. The firing of trigger can either enter the Debug Mode or raise a breakpoint exception, which depends on the trigger registers.

Table 57. Trigger Registers Implemented in Nios V Processor

Name	Registers	Description
`tselect`	Trigger Select	Nios V processor supports one trigger, therefore the processor selects Trigger 0 at default (value set at 0).
`tdata1`	Trigger Data 1	The value of `type` field is 2 to represent Trigger 0 as an address or data match trigger. Write behavior to `tdata` registers depends on the `dmode` field. The remaining bits acts as `mcontrol`.
`mcontrol`	Match Control	Controls address and data trigger implementation according to `action`, `m`, `execute`, `store` and `load` fields. The Nios V processor does not implement other fields.
`tdata2`	Trigger Data 2	Holds trigger-specific data (virtual address, instruction opcode, data stored or loaded).
`tinfo`	Trigger Info	The value of info field is 4 at default to signify `tdata1.type` is 2. This implies selected trigger is an address or data match trigger.

Based on the bit field setting in `mcontrol` register, Nios V processor can implement different trigger types after the selected trigger matches the `tdata2` register.

Table 58. Nios V Processor Triggers Condition and Firing Time

Triggers	Condition	Firing Time	Exception Program Counter
Instruction Address Trigger	Program Counter matches `tdata2`	Before executing the instruction	The processor sets the Machine Exception Program Counter (`mepc`) to the instruction address (PC).
Instruction Opcode Trigger	Instruction opcode matches `tdata2`	Before executing the instruction	
Store Address Trigger	Store address matches `tdata2`	After executing the store instruction	The processor sets the `mepc` to the next instruction address (PC + 4).
Store Data Trigger	Store data matches `tdata2`	After executing the store instruction	
Load Address Trigger	Load address matches `tdata2`	After executing the load instruction	
Load Data Trigger	Load data matches `tdata2`	After executing the load instruction	

Table 59. Address or Data Match Trigger Type

Trigger	mcontrol bit fields			
	select	execute	store	load
Instruction Address	0	1	0	0
Instruction Opcode	1	1	0	0
Store Address	0	0	1	0
Store Data	1	0	1	0
Load Address	0	0	0	1
Load Data	1	0	0	1

Note: The trigger is disabled in the following situations:

- The Nios V processor is in debug mode.
- The Nios V processor is in machine mode, while `mcontrol.m` or `mcontrol.type` is 0.

3.3.10.4. Abstract Commands in Debug Mode

Nios V/g processor implements Access Register abstract command. The Access Register command allows read-write access to the processor registers including GPRs, CSRs, FP registers and Program Counter. The Access Register also allows program execution from program buffer. The debugger executes Access Register commands by writing into Abstract Command (`command`) register using the Access Register command encoding.

Table 60. Access Register Command Encoding

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
cmdtype								0	aarsize			aarpostincrement	postexec	transfer	write
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
regno															

Table 61. Fields Descriptions

Field	Role
`cmdtype`	Determine command type 0 : Indicates Access Register command.
`aarsize`	Specifies size of register access 2: Access the lowest 32 bit of register 3: Access the lowest 64 bit of register 4: Access the lowest 128 bit of register
`aarpostincrement`	0: No effect 1: regno is incremented after successful register access
`postexec`	0: No effect 1: Execute program in program buffer

continued...

Field	Role
transfer	Acts in conjunction with write field. 0: Ignore value in write field 1: Execute operation specified by write field.
write	0: Copy data from register 1: Copy data to register
regno	Register address to be accessed.

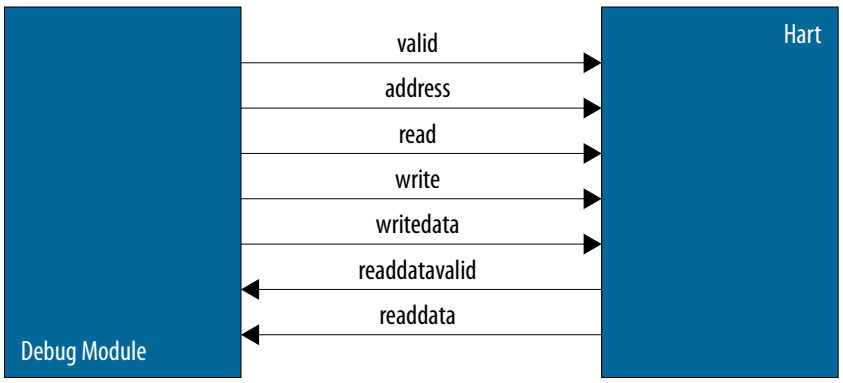
Note: The Nios V/g processor does not support abstract commands when hardware thread is not halted.

Table 62. Software Response to Unimplemented Commands

Field	State	
cmderr[10:8]	0	No error
	1	Busy
	2	Command not supported
	3	Exception - from program buffer instruction
	4	Command not executed because hart unavailable, or not in correct state to execute command.
	5	Abstract command failed due to bus error
	6	RSVD
	7	Command failed for other reasons.

Debug Module has the Abstract Control and Status CSR which includes the `cmderr` field. The `cmderr` field represents the current state of abstract command being executed. If the `cmderr` field is non-zero, writes to the `command` register are ignored. To clear the `cmderr` field, write 1 for every bit in the field.

Figure 9. Debug Module Interface Signals



Avalon memory-mapped interface implement the Register Access using request/response bus with Debug Module being the initiator and core being the responder. Address bus carries the register ID.

3.3.10.5. Hardware/Software Interface

Debug Module read or write to Debug Module registers, which initiate the interaction with the debugger. Each register has a fixed address as specified in the RISC-V Debug Support specification. Debugger can determine the register implementation status by write or read from the Debug Module registers. Unimplemented registers return 0 when read.

Debugger can check the status of the system by reading Debug Module Status (`dmstatus`) register. Debug Module Status register is read-only and provides status of the Debug Module and the selected harts.

You can access a specific hart by writing to the `hartsel` field in `dmcontrol`. Other fields in `dmcontrol` specify the action a debugger can take.

Halt Summary 0 (`haltsum0`) register reflects the status of a hart (halted/not halted). The LSB of this register can reflect whether hart is halted or not. Other bits is always 0. This is a read-only register of the debugger.

Related Information

RISC-V Debug Release
More information about the conceptual view of the states, refer to Section : Overview of States, Figure: Run/Halt Debug State Machine for single hart systems.

3.4. Programming Model

3.4.1. Privilege Levels

The privilege levels in Nios V/g processor are designed based on the RISC-V architecture specification. The privilege levels available are Machine Mode (M-mode) and Debug Mode (D-mode).

Related Information

The RISC-V Instruction Set Manual Volume II: Privileged Architecture Version 1.11

3.4.1.1. Machine Mode (M-mode)

The default operating mode is Machine mode (M-mode). The core boots up into M-mode after power-on reset by default. The M-mode provides low-level access to the machine implementation and all CSRs.

3.4.1.2. Debug Mode (D-mode)

The Debug mode (D-mode) is an additional privilege level to support off-chip debugging and manufacturing test.

The core can enter D-mode using any of the following methods:

- After reset when bit is set in debug CSR.
- Using debug interrupt.
- Using EBREAK instruction when bit is set in debug CSR.
- Using single step.

3.4.2. Control and Status Registers (CSR) Mapping

Control and status registers report the status and change the behavior of the processor. Since the processor core only supports M-mode and D-mode, Nios V/g processor implements the CSRs supported by these two modes.

Table 63. Control and Status Registers List

Number	Privilege	Name	Description
Machine Information Register			
0xF11	MRO	mvendorid	Vendor ID. Refer to Vendor ID Register Fields.
0xF12	MRO	marchid	Architecture ID. Refer to Architecture ID Register Fields.
0xF13	MRO	mimpid	Implementation ID. Refer to Implementation ID Register Fields.
0xF14	MRO	mhartid	Hardware thread ID. Refer to Hardware Thread ID Register Fields.
Machine Trap Setup			
0x300	MRW	mstatus	Machine status register. Refer to Machine Status Register Fields.
0x301	MRW	misa	ISA and extensions. Refer to Machine ISA Register Fields.
0x304	MRW	mie	Machine interrupt-enable register. Refer to Machine Interrupt-Enable Register Fields.
0x305	MRW	mtvec	Machine trap-handler base address. Refer to .Machine Trap-Handler Base Address Register Fields.
Machine Trap Handling			
0x341	MRW	mepc	Machine exception program counter. Refer to Machine Exception Program Counter Register Fields.
0x342	MRW	mcause	Machine trap cause. Refer to Machine Trap Cause Register Fields.
0x343	MRW	mtval	Machine bad address or instruction. Refer to Machine Trap Value Register Fields.
0x344	MRW	mip	Machine interrupt pending. Refer to Machine Interrupt-Pending Register Fields.
Debug Mode Registers			
0x7B0	DRW	dcsr	Debug control and status register. Refer to Debug Control and Status Register Fields.
0x7B1	DRW	dpc	Debug Program Counter. Refer to Debug Program Counter Register Fields.
Trigger Registers			
0x7A0	MRW	tselect	Trigger select. Refer to Trigger Select Register Fields.
0x7A1	MRW	tdata1 (mcontrol)	Trigger data 1 (Match Control). Refer to Trigger Data 1 (Match Control) Register Fields.
0x7A2	MRW	tdata2	Trigger data 2. Refer to Trigger Data 2 Register Fields.
0x7A4	MRO	tinfo	Trigger info. Refer to Trigger Info Register Fields.

Related Information

The RISC-V Instruction Set Manual Volume II: Privileged Architecture
More information about M-mode CSR and their respective fields.

3.4.2.1. Control and Status Register Field

The value in the each CSR registers determines the state of the Nios V/g processor. The field descriptions are based on the RISC-V specification.

Table 64. Vendor ID Register Fields

The `mvendorid` CSR is a 32-bit read-only register that provides the JEDEC manufacturer ID of the provider of the core.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bank															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bank									Offset						

Table 65. Architecture ID Register Fields

The `marchid` CSR is a 32-bit read-only register encoding the base microarchitecture of the hart.

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
Architecture ID														

Table 66. Implementation ID Register Fields

The `mimpid` CSR provides a unique encoding of the version of the processor implementation.

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
Implementation														

Table 67. Hardware Thread ID Register Fields

The `mhartid` CSR is a 32-bit read-only register that contains the integer ID of the hardware thread running the code

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
Hart ID														

Table 68. Machine Status Register Fields

The `mstatus` register is a 32-bit read-write register that keeps track of and controls the hart's current operating state.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SD	WPRI								TSR	TW	TVM	MXR	SUM	MPRV	XS[1:0]
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
XS[1:0]	FS[1:0]		MPP[1:0]		WPRI		SPP	MPIE	WPRI	SPIE	UPIE	MIE	WPRI	SIE	UIE

Table 69. Machine ISA Register Fields

The `misa` CSR is a read-write register reporting the ISA supported by the hart.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MXL[1:0]		WLRL				Extension[25:0]									
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Extension[25:0]															

Table 70. Machine Interrupt-Enable Register Fields

The `mie` register is a 32-bit read-write register that contains interrupt enable bits.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WPRI															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WPRI				MEIE	WPRI	SEIE	UEIE	MTIE	WPRI	STIE	UTIE	MSIE	WPRI	SSIE	USIE

Table 71. Machine Trap-Handler Base Address Register Fields

The `mtvec` register is a 32-bit read/write register that holds trap vector configuration, consisting of a vector base address (BASE) and a vector mode (MODE).

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Base[31:2]															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Base[31:2]														Mode	

Table 72. Machine Exception Program Counter Register Fields

The `mepc` register is a 32-bit read-write register that holds the addresses of the instruction that was interrupted or that encountered the exception when a trap is taken into M-mode.

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
mepc														

Table 73. Machine Trap Cause Register Fields

The `mcause` register is a 32-bit read-write register that hold the code indicating the event that caused the trap when a trap is taken into M-mode.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Interrupt	Exception code [30:16]														
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Exception code [15:0]															

Table 74. Machine Trap Value Register Fields

The `mtval` register is a 32-bit read-write register that is written with exception-specific information to assist software in handling the trap when a trap is taken into M-mode.

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
mtval														

Table 75. Machine Interrupt-Pending Register Fields

The `mip` register is a 32-bit read/write register containing information on pending interrupts.

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WPRI															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WPRI				MEIP	WPRI	SEIP	UEIP	MTIP	WPRI	STIP	UTIP	MSIP	WPRI	SSIP	USIP

Table 76. Trigger Select Register Fields

The `tselect` register is a 32-bit read/write register that selects the current trigger is accessible by other trigger register.

Bit Field														
31	30	29	28	27	26	25	...	6	5	4	3	2	1	0
`tselect`														

Table 77. Trigger Data 1 (Match Control) Register Fields

The `tdata1 (mcontrol)` register is a 32-bit read/write register containing information on the trigger type, tdata registers accessibility, and trigger implementation

Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
type=2				dmode	maskmax						hit	select	timing	sizelo	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
action				chain	match				m	0	s	u	execute	store	load

Table 78. Trigger Data 2 Register Fields

The `tdata2` register is a 32-bit read/write register containing the trigger-specific data.

Bit Field														
31	30	29	28	27	26	25	…	6	5	4	3	2	1	0
tdata2														

Table 79. Trigger Info Register Fields

The `tinfo` register is a 32-bit read-only register containing information on each possible tdata1.type.

Bit Field												
31	30	29	…	18	17	16	15	14	13	…	2	1	0
0							info						

Table 80. Debug Control and Status Register Fields

The `dcsr` CSR is a 32-bit read-write register containing information and status during D-mode

Bit Field														
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
debugver				0											
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ebreakm	0	ebreaks	ebreaku	stepie	stopcount	stoptime	cause			0	mprven	nmip	step	prv	

Table 81. Debug Program Counter Register Fields

Upon entry to D-mode, `dpc` CSR is updated with the virtual address of the next instruction to be executed.

Bit Field														
31	30	29	28	27	26	25	…	6	5	4	3	2	1	0
dpc														

3.5. Core Implementation

3.5.1. Instruction Set Reference

The Nios V/g processor is based on the RV32IMA specification, and there are 6 types of instruction formats. They are R-type, I-type, S-type, B-type, U-type, and J-type.

Table 82. Instruction Formats (R-type)

Bit Field (R-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
funct7							rs2					rs1			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs1	funct3			rd			opcode								

Table 83. Instruction Formats (I-type)

Bit Field (I-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
imm[11:0]												rs1			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs1	funct3			rd					opcode						

Table 84. Instruction Formats (S-type)

Bit Field (S-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
imm[11:5]							rs2					rs1			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs1	funct3			imm[4:0]					opcode						

Table 85. Instruction Formats (B-type)

Bit Field (B-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
imm[12]	imm[10:5]						rs2					rs1			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs1	funct3			imm[4:1]			imm[11]		opcode						

Table 86. Instruction Formats (U-type)

Bit Field (U-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
imm[31:16]															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
imm[15:12]				rd					opcode						

Table 87. Instruction Formats (J-type)

Bit Field (J-type)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
imm[20]	imm[10:1]										imm[11]	imm[19:16]			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
imm[15:12]				rd					opcode						

4. Nios V Processor Reference Manual Archives

For the latest and previous versions of this user guide, refer to Nios® V Processor Reference Manual. If an IP or software version is not listed, the user guide for the previous IP or software version applies.

IP versions are the same as the Intel Quartus Prime Design Suite software versions up to v19.1. From Intel Quartus Prime Design Suite software version 19.2 or later, IP cores have a new IP versioning scheme.

5. Document Revision History for the Nios V Processor Reference Manual

Document Version	Intel Quartus Prime Version	Changes
2023.05.26	23.1	Added a link to *AN 980: Nios V Processor Intel Quartus Prime Software Support.*
2023.04.14	23.1	• Updated *Nios V/m Processor Performance Benchmarks*. • Added a new section *Nios V/g Processor*. • Updated product family name to "Intel Agilex 7".

Document Version	Intel Quartus Prime Version	IP Version	Changes
2022.10.31	22.1std	1.0.0	• Updated references from *Intel Quartus Prime Pro Edition* to Intel Quartus Prime to indicate support for both Pro and Standard Edition. • Added Table: *Nios V/m Processor Performance Benchmarks in Intel FPGA Devices for Intel Quartus Prime Standard Edition* in *Processor Performance Benchmarks* section.
2022.09.26	22.3	22.3.0	• Updated the values in Table: *Nios V/m Processor Performance Benchmarks in Intel FPGA Devices*. • Replaced Table: *Reset and Debug Signals* with new signals, types, and descriptions. • Updated the step to set exception address in section *Exception Controller*.
2022.08.01	22.2	21.3.0	• Edited the performance metric value in Table: *Architecture Performance*. • Added new sections — *Trigger* — *Typical Use Cases* • Edited the following sections: — *Reset and Debug Signals* — *RISC-V based Debug Module* — *Debug Mode* — *Halt from Debug Module* — *Control and Status Registers (CSR) Mapping* — *Control and Status Register Field*
2022.06.30	22.1	21.2.0	Added new section *Reset and Debug Signals*.
2022.03.28	21.4	21.1.1	Updated *RISC-V based Debug Module* section with details for Nios V processor.
2021.12.13	21.4	21.1.1	Updated IP version and Intel Quartus Prime version.
2021.11.15	21.3	21.1.0	Edited Table: *Architecture Performance* in Section: *Processor Performance Benchmarks*.

continued...

Document Version	Intel Quartus Prime Version	IP Version	Changes
			• Change *CoreMark* to *CoreMark/MHz Ratio* and updated the value to *0.32148*.
2021.10.04	21.3	21.1.0	Initial release.